Lecture 14 Compendium: Training AI Models for Pest Detection

Module 4: Practical Skills & Applied AI

Dr. Kadeghe Fue
PhD Agroecology

Compendium for Advanced Study

Contents

1 The AI Training Lifecycle **2**

2 Frameworks: PyTorch vs. TensorFlow **2**
2.1 PyTorch 2
2.2 TensorFlow / Keras 2

3 Transfer Learning: The Engine of Ag-AI **2**
3.1 The Theory 3
3.2 Implementation Strategy 3

4 Mathematical Foundations: Loss Functions **3**
4.1 Cross-Entropy Loss 3
4.2 Focal Loss (For Imbalanced Data) 3

5 The Training Loop (Python Implementation) **4**

6 Evaluation Metrics **5**
6.1 Confusion Matrix 5
6.2 Derived Metrics 5

7 Hyperparameter Tuning **5**
7.1 Strategies 5

8 Deployment: Model Compression **6**
8.1 Quantization 6
8.2 Pruning 6

9 Review Questions & Assignment **6**

1 The AI Training Lifecycle

Training an Artificial Intelligence model is not merely a coding exercise; it is an experimental science. For a PhD student, the goal is not just to achieve high accuracy, but to understand *why* the model converges (or diverges) and to prove its robustness.

The lifecycle consists of four distinct phases:

1. **Setup:** Environment configuration (GPU, Frameworks) and Data Loading.
2. **Architecture Design:** Selecting the backbone (ResNet, EfficientNet) and modifications (Transfer Learning).
3. **Training Loop:** The mathematical optimization process (Forward/Backward pass).
4. **Evaluation & Tuning:** Metric calculation and Hyperparameter optimization.

The Research Standard

Academic rigor demands reproducibility. "I ran the code and it worked" is insufficient. You must document the Hyperparameters, Seed values, and Hardware used (e.g., NVIDIA T4 GPU) so that another researcher can replicate your results exactly.

2 Frameworks: PyTorch vs. TensorFlow

While there are many libraries, academic research is currently dominated by **PyTorch**.

2.1 PyTorch

- **Paradigm:** Dynamic Computational Graph (Define-by-Run). This allows for easy debugging and dynamic architecture changes (e.g., variable input sizes).
- **Adoption:** Used in $> 75\%$ of CVPR/ICCV research papers.
- **Pros:** Pythonic, intuitive for researchers.

2.2 TensorFlow / Keras

- **Paradigm:** Static Graph (Define-and-Run).
- **Adoption:** Industry standard for production deployment.
- **Pros:** Excellent deployment tools (TFLite, TFServing), but harder to debug custom architectures.

Note: This course utilizes PyTorch for its flexibility in research contexts.

3 Transfer Learning: The Engine of Ag-AI

Training a Deep Convolutional Neural Network (CNN) from scratch requires millions of images and weeks of GPU time. In Agriculture, we use **Transfer Learning**.

3.1 The Theory

A CNN learns hierarchical features:

- **Layer 1-3:** Edges, Corners, Colors (Generic features).
- **Layer 4-N:** Textures, Object Parts, Semantic concepts (Specific features).

A model trained on *ImageNet* (14 million images of cats, cars, etc.) has already learned excellent generic feature extractors. We can repurpose these "eyes" to look at maize leaves.

3.2 Implementation Strategy

1. **Load Pre-trained Weights:** Initialize the network with ImageNet weights.
2. **Freeze Backbone:** Set `requires_grad = False` for the convolutional layers. This prevents the gradients from destroying the learned filters.
3. **Replace Head:** Swap the final Fully Connected (FC) layer. ImageNet has 1000 classes; we usually have 2-10 (e.g., Healthy, Blight, Spot).
4. **Fine-Tune:** Train only the new FC layer (warm-up), then optionally unfreeze the last few blocks of the backbone and train with a very low learning rate ($1e^{-5}$).

4 Mathematical Foundations: Loss Functions

The Loss Function $L(\hat{y}, y)$ quantifies the error between the prediction $\hat{y}$ and the ground truth y. The optimizer minimizes this value.

4.1 Cross-Entropy Loss

The standard for classification.

$$CE = -\sum_{c=1}^{C} y_c \log(\hat{y}_c) \tag{1}$$

4.2 Focal Loss (For Imbalanced Data)

Agricultural datasets are often imbalanced (few diseases, many healthy). Cross-Entropy can be overwhelmed by the "easy" healthy examples. **Focal Loss** adds a modulating factor to down-weight easy examples and focus on hard ones.

$$FL(p_t) = -(1 - p_t)^{\gamma} \log(p_t) \tag{2}$$

Where:

- p_t: The model's estimated probability for the class.
- γ: Focusing parameter (usually $\gamma = 2$). If a sample is easy ($p_t \approx 0.9$), the term $(1-0.9)^2 = 0.01$, significantly reducing the loss contribution.

5 The Training Loop (Python Implementation)

Below is a robust implementation of a training loop in PyTorch, including validation tracking and model checkpointing.

```python
import torch
import copy
import time

def train_model(model, dataloaders, criterion, optimizer, num_epochs=25):
    since = time.time()

    val_acc_history = []
    best_model_wts = copy.deepcopy(model.state_dict())
    best_acc = 0.0

    device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
    model = model.to(device)

    for epoch in range(num_epochs):
        print(f'Epoch {epoch}/{num_epochs - 1}')
        print('-' * 10)

        # Each epoch has a training and validation phase
        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()  # Set model to training mode
            else:
                model.eval()   # Set model to evaluate mode

            running_loss = 0.0
            running_corrects = 0

            # Iterate over data
            for inputs, labels in dataloaders[phase]:
                inputs = inputs.to(device)
                labels = labels.to(device)

                # Zero the parameter gradients
                optimizer.zero_grad()

                # Forward
                # Track history only if in train
                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    loss = criterion(outputs, labels)
                    _, preds = torch.max(outputs, 1)

                    # Backward + Optimize only if in training phase
                    if phase == 'train':
                        loss.backward()
                        optimizer.step()

                # Statistics
                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds == labels.data)

            epoch_loss = running_loss / len(dataloaders[phase].dataset)
            epoch_acc = running_corrects.double() / len(dataloaders[phase].
    dataset)

            print(f'{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')

```

```
            # Deep copy the model if it's the best so far
            if phase == 'val' and epoch_acc > best_acc:
                best_acc = epoch_acc
                best_model_wts = copy.deepcopy(model.state_dict())
            if phase == 'val':
                val_acc_history.append(epoch_acc)

    time_elapsed = time.time() - since
    print(f'Training complete in {time_elapsed // 60:.0f}m {time_elapsed % 60:.0
    f}s')
    print(f'Best Val Acc: {best_acc:4f}')

    # Load best model weights
    model.load_state_dict(best_model_wts)
    return model, val_acc_history
```

Listing 1: PyTorch Training Loop

6 Evaluation Metrics

In PhD research, stating "Accuracy: 95%" is meaningless without context. We must use robust metrics.

6.1 Confusion Matrix

A table showing True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN).

6.2 Derived Metrics

- **Precision:** $\frac{TP}{TP+FP}$. (Relevance). High precision means few false alarms (spraying healthy crops).
- **Recall (Sensitivity):** $\frac{TP}{TP+FN}$. (Completeness). High recall means few missed diseases.
- **F1-Score:** The harmonic mean. $2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$.

The Cost Function of Agriculture

Which is worse? **False Positive (FP):** You spray a healthy plant. Cost = Chemical + Fuel. **False Negative (FN):** You miss a diseased plant. Cost = Potential outbreak/Total crop loss. Usually, maximizing **Recall** is more important in epidemiology to prevent spread.

7 Hyperparameter Tuning

How do you choose the Learning Rate, Batch Size, or Dropout rate?

7.1 Strategies

1. **Grid Search:** Try every combination. (Computationally expensive).
2. **Random Search:** Try random combinations. (Surprisingly effective).
3. **Bayesian Optimization:** Uses a probabilistic model to predict which parameters will work best based on previous trials. (The PhD Standard).

Learning Rate Schedulers: Instead of a fixed LR, we decay it. * **Step Decay:** Drop LR by factor of 0.1 every 7 epochs. * **Cosine Annealing:** Smoothly decrease LR following a cosine curve.

8 Deployment: Model Compression

Training a ResNet50 results in a 100MB file. This is too large for an Arduino or a rural 2G connection.

8.1 Quantization

Neural networks typically use 32-bit Floating Point numbers (Float32). **Quantization** converts weights to 8-bit Integers (Int8).

- **Size Reduction:** 4× smaller.
- **Speedup:** 2× to 4× faster on CPU (using NEON instructions).
- **Accuracy Drop:** Usually negligible ($< 1\%$).

8.2 Pruning

Removing neurons or connections with weights close to zero, as they contribute little to the output. This creates "Sparse Matrices."

9 Review Questions & Assignment

1. **Architecture:** Explain why we freeze the backbone layers during the first epoch of Transfer Learning. What happens to the pre-trained weights if we don't?
2. **Math:** Calculate the Focal Loss for a sample where the predicted probability $p_t = 0.9$ and $\gamma = 2$. Compare this to standard Cross-Entropy. How does this help with "Easy" examples?
3. **Design:** You are deploying a pest detection model to a drone powered by a Raspberry Pi. The standard ResNet model runs at 2 FPS. What specific techniques (Quantization, Distillation, Architecture search) would you use to reach 10 FPS?

References

1. Lin, T. Y., et al. (2017). *Focal Loss for Dense Object Detection.* ICCV.
2. Goodfellow, I., et al. (2016). *Deep Learning.* MIT Press.
3. PyTorch Documentation. `https://pytorch.org/docs/stable/index.html`